

ADVICE DOCUMENT

Written by
Ayomide Olagoke
Paul Demenko

Emmen, Netherlands
Version 1.0

Table of Contents

Version Control	3
Introduction	4
Design Patterns	4
Factory Method	4
Comparison with Builder Pattern	4
Comparison with Abstract Factory	5
Command.....	6
Comparison with State Pattern	6
Comparison with Mediator Pattern	6
Additional Motivations	7
Composite.....	8
Justification for Using Composite Pattern	8
Comparison with Decorator Pattern	8
Comparison with Flyweight Pattern	8
Additional Motivation	9
Refactoring.....	10
DTAP.....	10

Version Control

Version	Date	Content	Editor(s)
1.0	14/02/2024	First draft	Ayomide Olagoke
1.1	07/04/2024	Second draft	Ayomide Olagoke Paul Demenko
1.2	13/06/2024	Added proper justification to the design patterns chosen	Ayomide Olagoke Paul Demenko

Introduction

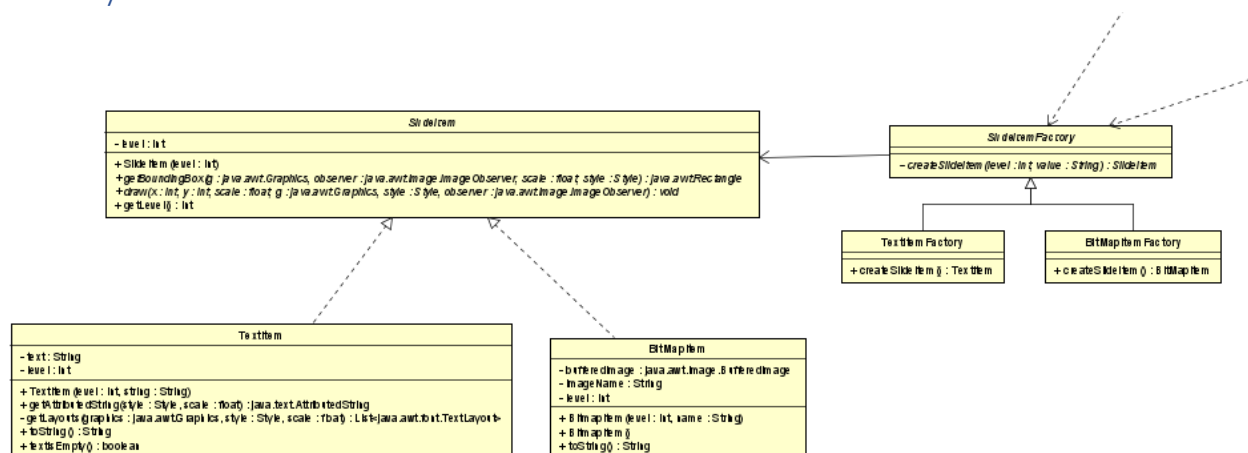
This document contains an in-depth explanation of design patterns used to improve the JabberPoint program, issues found in the initial version of JabberPoint programme, and the refactoring done to improve the readability and structure of the code.

We created UML diagrams to analyse the provided version of JabberPoint. We created an activity diagram to understand how JabberPoint runs, a class diagram to view the architecture of its code, a sequence diagram to show the process of running Jabber point, and a use-case diagram showing the functionalities expected by a user. We created new versions of these diagrams to give an overview of our proposed solution.

Design Patterns

We applied two design patterns and noticed the presence of one design pattern in the program. We applied the factory method, creational design pattern, and command, a behavioural design pattern.

Factory Method



We applied the Factory Method pattern to ease the creation of slide items due to several advantages it offers over other creational patterns. This design pattern reduces the code required to construct a new slide item by abstracting the details of creation and encapsulating the creation logic within separate factory classes (`TextItemFactory` and `BitmapItemFactory`).

Comparison with Builder Pattern

1. **Simpler Object Creation:** Builder pattern is useful for creating complex objects step-by-step. However, our slide items are simple objects, requiring level position and one additional parameter – either the name of the file for `BitmapItem` or the text to be

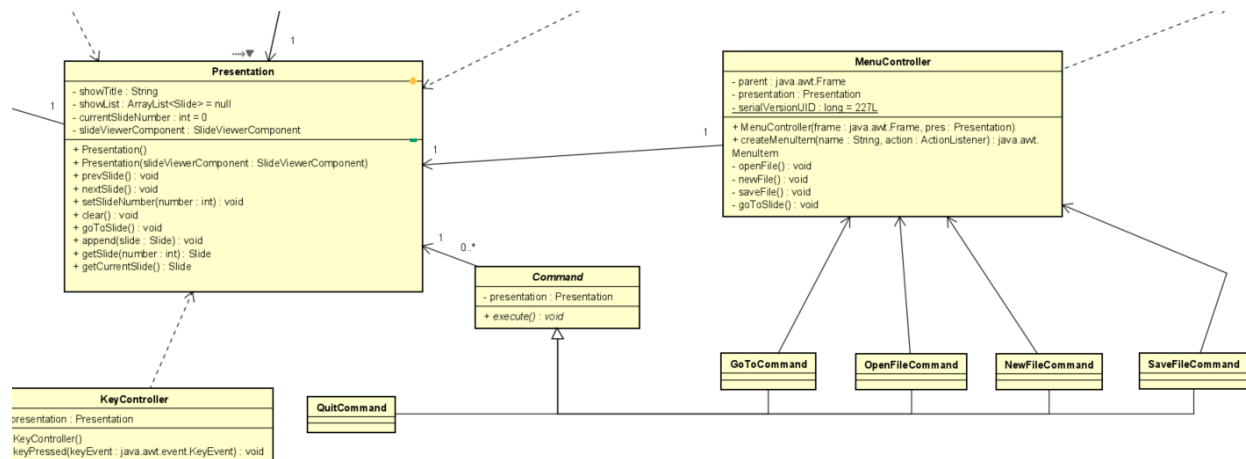
displayed for `TextItem`. Factory Method pattern provides a simpler and more direct approach to object creation.

2. **Simplicity:** While Builder pattern provides greater control over the construction process, Factory Method pattern suffices our requirements. It separates the creation logic into distinguished factory classes, making the code more maintainable and lowering complexity.
3. **Ease of Adding New Types:** Factory Method pattern allows for easy addition of new slide item types without modifying existing code (thus, adhering to Open/Closed principle).

Comparison with Abstract Factory

1. **Reduced Complexity:** Although the Abstract Factory pattern can be applied in this case, it introduces unnecessary complexity by requiring a family of related products. Since our requirement is to create individual slide items rather than families of related objects, the Factory Method pattern is a more straightforward and efficient choice.
2. **Single Responsibility:** The Factory Method pattern keeps the responsibility of creating individual slide items within their respective factory classes, making the codebase easier to manage and understand compared to the broader scope of the Abstract Factory.

Command



We applied the Command pattern to the initial design of the menu controller. We did it to turn every request into a standalone object, which gets carried out whenever a request is called. Instead of the `MenuController` containing the logic for all buttons and shortcuts related to requests, we created a `Command` abstract class with a method `execute()` that gets implemented by the `Command` child classes. Using this design pattern promotes loose coupling and makes it easier to extend the system of requests. The support of asynchrony, flexibility, and efficient handling of user requests makes this design pattern a better choice over State or Mediator design patterns.

Comparison with State Pattern

1. **Single Responsibility:** State pattern is primarily used to alter behaviour of an object when its state changes. Our requirement involves encapsulating all user requests (such as opening and saving the presentation, navigating through slides) into standalone objects, making Command pattern more applicable.
2. **Request Encapsulation:** Command pattern is effective at encapsulating requests as individual objects, ensuring a separation between the sender and receiver. In contrast, using State pattern in this scenario would introduce unnecessary complexity into the state management of `MenuController`.

Comparison with Mediator Pattern

1. **Direct Communication:** While Mediator pattern centralises interactions to reduce direct communication between objects, our use case benefits more from Command pattern. By encapsulating user actions as commands, we centralise the commands and enable different classes to perform actions without directly interacting with `Presentation` or other specific classes.
2. **Decoupling Requests:** The Command pattern promotes loose coupling by decoupling the object that invokes the operation from the one that knows how to perform it. Mediator pattern, on the other hand, centralises control and would not provide the same level of encapsulation for individual requests.

Additional Motivations

1. **Flexibility and extensibility:** Command pattern allows us to easily add new commands without modifying existing code, adhering to Open/Closed principle. Each command is self-contained and implements the `execute()` method, making the system flexible and extensible.
2. **Simplified MenuController:** Instead of `MenuController` containing the logic for all buttons and shortcuts, this logic is encapsulated within individual command classes. This simplifies `MenuController` and makes the codebase more modular and maintainable.

Composite

We propose applying the Composite pattern to manage the `Slide` and `SlideItem` hierarchy in the system. This pattern is beneficial for several reasons. It allows individual objects and compositions of objects to be treated uniformly, which simplifies the client code and enhances scalability.

Justification for Using Composite Pattern

1. **Uniformity:** Composite pattern allows both individual objects (`SlideItems`) and compositions of objects (`Slides`) to be treated uniformly. This simplifies the client code, as it can interact with all objects through a consistent interface.
2. **Scalability:** Composite pattern eases the addition of new types of `SlideItems` without altering the code that uses them. This aligns with Open/Closed Principle, enabling the system to scale and adapt over time.
3. **Hierarchy Management:** Using Composite pattern provides a structured approach to managing the hierarchy of `SlideItems`. It allows for better organisation and control of different items that make up a slide.
4. **Extensibility:** Composite pattern makes it straightforward to introduce new types of `SlideItems`. This extensibility ensures the system can evolve by incorporating new functionalities and features without disrupting existing code.

By applying Composite pattern, we enhance system's ability to handle complex relationships between `Slides` and `SlideItems` and ensure future scalability and extensibility.

Comparison with Decorator Pattern

1. **Dynamic Behaviour Addition:** Decorator Pattern allows behaviour to be added to individual objects dynamically without affecting the behaviour of other objects from the same class. It focuses on extending functionality through composition rather than inheritance while Composite Pattern focuses on creating tree structures of objects to represent part-whole hierarchies. It treats individual objects and compositions uniformly, which simplifies client code.
2. **Single Responsibility Principle:** Decorator pattern follows the Single Responsibility Principle by assigning specific functionality to different classes. This approach may result in many small classes. Similarly, Composite pattern adheres to the Single Responsibility Principle by isolating the management of part-whole hierarchies from the core responsibilities of individual objects.
3. **Use Case Complexity:** Decorator pattern can introduce complexity when managing combinations of decorators, particularly when multiple behaviours need to be added to an object. In contrast, the Composite pattern simplifies the structure by allowing composite objects to be treated the same way as individual objects are managed, making it more straightforward to manage hierarchical relationships.

Comparison with Flyweight Pattern

1. **Memory Efficiency:** Composite pattern does not primarily focus on memory efficiency, rather on managing part-whole hierarchies. It ensures uniform treatment of individual and composite objects while Flyweight pattern is used to minimise memory usage by sharing as much data as possible with similar objects. Composite

pattern is more applicable to our use case because it does not require high memory use and it would help us manage `Slide` and `SlideItem` hierarchy and streamline the client code.

2. **Shared State:** Flyweight Pattern uses intrinsic and extrinsic states to share data among multiple objects, reducing redundancy and this can lead to complexity in state management while Composite pattern manages hierarchical structures without the need for shared state. Each component maintains its state, simplifying the management of complex object relationships.
3. **Scalability and Structure:** Flyweight pattern scales well in terms of memory usage by sharing data. However, managing shared states can become complex while Composite pattern scales well in terms of structure by allowing complex hierarchies to be composed of simple objects. It provides a clear and straightforward way to manage these hierarchies.

Additional Motivation

1. **Consistency In Client Code:** The uniform treatment of individual and composite objects reduces complexity, making the system easier to maintain and extend.

Refactoring

The initial version of the code contained issues such as;

- Bad comments,
- Poor code structure,
- Classes such as `MenuController`, `Slide` and others with methods with high cohesion,
- Non-self-explanatory variables, method parameters, and method names.

We refactored the program to have understandable comments, applied design to promote the extension of certain functionalities, created methods to handle a single function in classes with high cohesion to reduce the present cohesion, added a util package that contains `Constants` class and a `ResourceAccessor` class that contains methods for loading files. We created unit tests to detect bugs and to ensure the program is working after refactoring. The commits are mostly self-explanatory, so every part of refactoring may be backtracked and understood.

DTAP

The initial git setup did not include any CI/CD or branching. We agreed to work on refactoring in a team, and we needed a setup to support that. The git repository consists of 3 branches – `main`, `test`, and `dev`.

`main` branch is used to release a stable version of the application. It has a pipeline that will automatically create a GitHub release whenever a pull request from `test` to `main` is merged.

`test` branch is running unit tests. It also runs code coverage and uploads the code coverage report to artifacts. The report is also hooked up to *CodeCov* to be inspected by the team. Sadly, the application cannot be run in a headless environment (and GitHub Actions do not have a connected display), so the report cannot be fully trusted. To evade this, the group is also submitting code coverage report generated by *IntelliJ*.

`dev` branch serves as the final endpoint for any feat branch. It runs basic compilation command to check that no broken code is merged. It also has a hook to a linter, `google-java-format`, that will check whether the files follow the Google's code conventions. If any mismatch is found, the pipeline will commit the changed files, so that only formatted files are present on `dev` branch.